

Day 2 Lab: Claude Code Fundamentals

Duration: 90 minutes | **Difficulty:** Beginner → Intermediate

Prerequisites: Claude Code installed, API key configured, Node.js 18+, Git installed

© 2026 AIA Copilot — Instructor-Led Training Material

Overview

This hands-on lab will take you from your first Claude Code session to building production-ready projects. You'll learn the core techniques that separate casual AI-assisted coding from professional agentic development.

What you'll build: - A REST API with proper error handling and validation - A CLI tool with watch mode and syntax highlighting - Clean Git workflows with conventional commits

What you'll master: - The agentic loop: how Claude plans, executes, and validates autonomously - CLAUDE.md: teaching Claude your team's conventions - Context control with @ mentions - Git workflows in natural language - Iterative development patterns

Exercise 1: Setup & First Magic Moment (15 min)

Goal

Experience the "magic moment" where Claude Code builds a working API from an empty directory.

Steps

1. Verify Installation

```
claude --version
```

Expected output: Claude Code CLI v1.x.x (or similar)

If not installed:

```
# Recommended (auto-updates)
curl -fsSL https://claude.ai/install.sh | bash

# Or via Homebrew
brew install --cask claude-code
```

2. Authenticate

```
claude login
```

This opens your browser for OAuth authentication. Follow the prompts, authorize the CLI, and return to your terminal.

Expected output: `✓ Successfully authenticated as [your-email]`

3. Create Project Directory

```
mkdir todo-api
cd todo-api
```

4. Start Interactive Session

```
claude
```

You're now in an interactive Claude Code session. The prompt changes to `claude>`.

5. The Magic Prompt

Copy and paste this exact prompt:

```
Create a simple Express API with these endpoints:
- GET /todos - list all todos
- POST /todos - create a new todo (title, completed)
- PUT /todos/:id - update a todo
- DELETE /todos/:id - delete a todo

Include proper error handling, input validation, and an in-memory store.
Initialize with npm and install dependencies.
```

What you'll see: - Claude analyzes the empty directory - Plans the project structure - Creates `package.json` with proper dependencies - Runs `npm install` automatically - Creates `src/index.js` with Express server - Implements all four endpoints - Adds validation and error handling - Starts the server (usually on port 3000)

6. Verify It Works

In a new terminal (keep Claude session running):

```
# List todos (should be empty array)
curl http://localhost:3000/todos

# Create a todo
curl -X POST http://localhost:3000/todos \
  -H "Content-Type: application/json" \
  -d '{"title": "Learn Claude Code", "completed": false}'

# List todos again (should show the new todo)
curl http://localhost:3000/todos
```

Expected output:

```
[]
{"id":1,"title":"Learn Claude Code","completed":false}
[{"id":1,"title":"Learn Claude Code","completed":false}]
```

What Just Happened?

This is the **agentic loop** in action:

1. **Planning:** Claude read your prompt, understood the requirements, and created a mental plan
2. **Execution:** It autonomously created files, ran npm commands, wrote code
3. **Validation:** It checked that the structure made sense before presenting it to you

You didn't tell Claude *how* to build it. You told it *what* you wanted. Claude: - Knew to use Express (industry standard for Node.js APIs) - Initialized npm with sensible defaults - Structured the code properly (server setup, route handlers, in-memory store) - Added error handling without being asked - Used best practices (async/await, proper HTTP status codes)

This is fundamentally different from autocomplete or code snippets. Claude understood the **intent** and delivered a **working system**.

Challenge

Back in the Claude session, type:

```
Add a GET /todos/:id endpoint that returns a single todo by ID. Return 404 if not found.
```

Watch Claude: 1. Read the existing code 2. Add the new endpoint following the same pattern 3. Include proper error handling 4. Test the endpoint

Verify:

```
curl http://localhost:3000/todos/1
curl http://localhost:3000/todos/999 # Should return 404
```

Exercise 2: CLAUDE.md Mastery (15 min)

Goal

Learn how CLAUDE.md makes Claude follow your team's conventions automatically.

Steps

1. Exit the Claude Session

In your Claude session:

```
/exit
```

You're back to your normal terminal.

2. Create CLAUDE.md

Create a file called `CLAUDE.md` in the `todo-api` directory:

```

cat > CLAUDE.md << 'EOF'
# Todo API Project

## Tech Stack
- Node.js with Express
- In-memory data store (no database yet)
- Jest for testing

## Coding Conventions
- Use async/await, never callbacks
- Use const/let, never var
- All endpoints return JSON with consistent format: { success: true, data: ... }
- Include try/catch on all route handlers
- Use descriptive error messages with appropriate HTTP status codes

## File Structure
- src/index.js - App entry point and server setup
- src/routes/ - Route handlers (one file per resource)
- src/middleware/ - Express middleware
- tests/ - Jest test files mirroring src/ structure

## Commands
- npm start: Start the server
- npm test: Run Jest tests
- npm run dev: Start with nodemon (hot reload)
EOF

```

3. Start Claude Again

```
claude
```

4. Test BEFORE CLAUDE.md Awareness

First, clear Claude's memory of the previous session:

```
/clear
```

Now ask:

```
Add a PATCH /todos/:id/toggle endpoint that toggles the completed status
```

Observe carefully: - Does the response format match `{ success: true, data: ... }` ? - Does it use `async/await`? - Does it have `try/catch` error handling? - Is the error message descriptive?

You should see Claude following the conventions from CLAUDE.md even though you just cleared its memory!

5. Verify the Endpoint

```
# Create a todo first
curl -X POST http://localhost:3000/todos \
  -H "Content-Type: application/json" \
  -d '{"title": "Test toggle", "completed": false}'

# Toggle it
curl -X PATCH http://localhost:3000/todos/1/toggle

# Check the result
curl http://localhost:3000/todos/1
```

Expected output: The `completed` field should flip from `false` to `true`.

What Just Happened?

CLAUDE.md is magic. Here's why:

Every time Claude starts working, it automatically: 1. Looks for `CLAUDE.md` in the project root 2. Reads it before doing anything else 3. Treats it as the source of truth for your project

You wrote ONE file with your conventions, and now: - Every endpoint Claude creates follows your response format - Every function uses `async/await` - Every route has `try/catch` - File placement matches your structure

Without CLAUDE.md: You'd have to remind Claude every time: "Use `async/await`. Make sure error handling is consistent. Follow this response format..."

With CLAUDE.md: Claude just knows.

This is how professional teams use Claude Code. `CLAUDE.md` becomes your team's shared brain — new developers (human or AI) read it and instantly understand your standards.

Challenge

Add a new convention to `CLAUDE.md`:

```
## Coding Conventions
...existing conventions...
- All responses must include a `timestamp` field with the current ISO timestamp
```

Save the file, then ask Claude:

```
Add a GET /health endpoint that returns the server status
```

Check the response. Does it include a `timestamp` field? Claude should follow the new convention immediately!

Exercise 3: Context & @ Mentions (15 min)

Goal

Master context control with @ mentions to make Claude look exactly where you want.

Steps

1. Stay in the Claude Session

Keep your existing session running (don't `/exit`).

2. Practice Directory @ Mentions

Try these commands one at a time:

```
@src/routes/ What endpoints do we have and what's missing?
```

Watch Claude scan the routes directory and give you an inventory.

```
@package.json What dependencies do we have? Suggest any we might need for production.
```

Claude will read package.json and suggest things like helmet, cors, dotenv, etc.

```
@CLAUDE.md Are we following all conventions? Audit the codebase.
```

This is powerful: Claude uses CLAUDE.md as a checklist to audit your code!

3. Focused Context Exercise

Now we'll refactor code using precise @ mentions:

```
@src/routes/todos.js Refactor this file to separate validation logic into src/middleware/validate.js
```

Watch what happens: 1. Claude reads the specific file you mentioned 2. Extracts validation logic (checking for required fields, data types, etc.) 3. Creates a new file `src/middleware/validate.js` 4. Updates `todos.js` to import and use the middleware 5. Ensures the API still works

4. Context Window Exercise

Check your token usage:

```
/cost
```

Expected output: Something like `Total cost: $0.15 | Tokens: 12,450`

Now compress the conversation history:

```
/compact
```

Check again:

```
/cost
```

You should see fewer tokens but the same context awareness. Claude summarized the conversation and kept what matters.

5. Web @ Mentions

This is advanced but incredibly useful:

```
@https://expressjs.com/en/guide/error-handling.html Update our error handling to match Express best practices from this guide
```

Claude will: 1. Fetch the Express documentation 2. Read the error handling patterns 3. Update your code to match official best practices

What Just Happened?

@ mentions are surgical precision for Claude's attention.

Think of Claude's context window like working memory: - Without @ mentions: "Read the entire project and figure out what matters" - With @ mentions: "Read THIS file and do THIS thing"

Why it matters:

Large codebases: In a 50-file project, you don't want Claude reading everything. `@src/auth/login.js fix the password validation` focuses Claude's attention and saves tokens.

External docs: `@https://...` lets you bring in official docs, API references, or style guides on-demand.

Auditing: `@CLAUDE.md` turns your conventions file into an automated code reviewer.

The pattern: Be specific about context = better results + lower costs.

Challenge

Try using @ mentions with a URL to improve your code:

```
@https://github.com/goldbergonyi/nodebestpractices Review our error handling against this Node.js best practices guide and suggest 3 improvements
```

Watch Claude read the external resource and apply industry best practices to your code.

Exercise 4: Git Workflow with Claude Code (20 min)

Goal

Let Claude handle your entire Git workflow — from branching to commit messages to PR descriptions.

Steps

1. Initialize Git

Exit Claude (`/exit`) and run:

```
git init
git add -A
git commit -m "Initial commit: Todo API"
```

2. Start Claude and Make a Change

```
claude
```

Ask Claude:

```
Add request logging middleware that logs: timestamp, method, path, status code, and response time in milliseconds. Put it in src/middleware/logger.js and register it in index.js.
```

Watch Claude: - Create `src/middleware/logger.js` with logging logic - Use `console.log` with proper formatting - Register the middleware in `index.js` before routes - Test that it doesn't break existing endpoints

3. Let Claude Commit

Instead of writing a commit message yourself:

```
Create a descriptive commit message for these changes and commit them
```

Watch Claude: 1. Run `git status` to see what changed 2. Run `git diff` to see the actual changes 3. Write a commit message in conventional commit format 4. Run `git add` and `git commit`

Expected commit message format:

```
feat(middleware): add request logging middleware

- Created logger middleware that captures timestamp, method, path, status, and response time
- Registered logger in index.js before route handlers
- Logs in format: [timestamp] METHOD /path - STATUS (XXXms)
```

4. Branch Workflow

Now let's do a full feature branch workflow:

```
Create a new branch called 'feature/search', then add a GET /todos/search?q=term endpoint that searches todo titles. Commit when done.
```

Watch the magic: 1. `git checkout -b feature/search` 2. Implement the search endpoint 3. Test it works 4. `git add` and `git commit` with descriptive message

5. Generate PR Description

```
Write a PR description for the search feature branch comparing it to main
```

Expected output:

```
# Add Todo Search Functionality

## Overview
Implements a search endpoint that allows filtering todos by title.

## Changes
- Added GET /todos/search endpoint
- Accepts query parameter `q` for search term
- Case-insensitive partial matching on todo titles
- Returns filtered results in standard response format

## Testing
```bash
curl "http://localhost:3000/todos/search?q=learn"
```

## Review Notes

---

- Follows existing coding conventions from CLAUDE.md
- Includes error handling for missing query parameter
- Uses consistent response format

```
What Just Happened?

You just experienced Git workflow automation in natural language.

Traditional workflow:
```bash
git checkout -b feature/search
# Write code manually
git add .
git commit -m "add search endpoint" # Lazy message
# Open GitHub, click "New PR", write description manually
```

Claude Code workflow:

```
Create a feature branch, implement search, and commit it
```

Why this matters:

1. **No context switching:** Stay in natural language, don't drop to Git commands
2. **Better commit messages:** Claude writes conventional commits with descriptive bodies
3. **Automatic PR descriptions:** Claude generates professional PR descriptions by comparing branches

4. **Git best practices:** Claude follows proper workflows (feature branches, descriptive commits)

The deeper insight: Claude doesn't just write code. It understands the **entire development workflow** — planning, coding, testing, version control, documentation.

Challenge

Ask Claude to do a code review:

```
Review the diff between main and feature/search. Suggest any improvements to the
search implementation.
```

Claude will: - Run `git diff main..feature/search` - Analyze the changes - Suggest improvements (edge cases, performance, error handling)

Exercise 5: Real-World Mini-Project (25 min)

Goal

Build a complete CLI tool from scratch using everything you've learned.

Steps

1. Create New Project

Exit Claude and create a fresh directory:

```
mkdir ~/md-converter
cd ~/md-converter
```

2. Start Claude

```
claude
```

3. Create CLAUDE.md First

This is **best practice**: Define conventions before writing code.

```
Create a CLAUDE.md for a CLI tool project: Node.js, uses Commander for CLI, marked for
markdown parsing, highlight.js for syntax highlighting. Test with Jest. Follow conventi
onal commits.
```

Claude will generate a CLAUDE.md with: - Tech stack documentation - CLI conventions (argument parsing, error handling) - File structure for CLI tools - Testing guidelines

4. Build the CLI Tool

Now the big prompt:

```
Build a CLI tool called 'mdx' that converts markdown files to HTML with syntax highligh
ting. Requirements:
- Accept input file path as argument
- Accept optional --output flag for output path (default: same name with .html)
- Accept --theme flag (default, github, monokai)
- Include proper error handling for missing files
- Add --watch flag that watches for changes and auto-converts
- Create a sample markdown file with code blocks to test with
```

What you'll see: 1. `package.json` with commander, marked, highlight.js, and chokidar (for watch) 2. `src/cli.js` - Entry point with argument parsing 3. `src/converter.js` - Markdown → HTML conversion logic 4. `src/themes.js` - Theme definitions 5. `sample.md` - Test file with code blocks 6. npm install runs automatically 7. Proper error handling for missing files, invalid themes, etc.

5. Test It

```
node src/cli.js sample.md --theme github
```

You should see `sample.html` created with syntax-highlighted code blocks!

6. Iterate - Add Template Flag

```
Add a --template flag that wraps output in a full HTML page with CSS
```

Claude will: - Update the CLI argument parsing - Create an HTML template with proper `<head>`, styling, etc. - Modify the converter to use the template when flag is set - Update help text

7. Fix Watch Mode Bug

```
The watch mode crashes if the file is deleted while watching – fix it
```

Watch Claude: - Add error handling in the watch callback - Handle `ENOENT` errors gracefully - Maybe add a message like "File deleted, watching stopped"

8. Add Tests

```
Add unit tests for the markdown parsing and theme selection
```

Claude creates: - `tests/converter.test.js` with Jest tests - Mocks for file system operations - Tests for edge cases (missing files, invalid themes) - Updates package.json with test script

Run them:

```
npm test
```

9. Final Cleanup

```
Review the entire project. Clean up any issues, add JSDoc comments to all exported functions, update the README with usage examples, and create a clean commit history.
```

Claude will: - Audit the codebase for issues - Add JSDoc comments - Create/update README with installation and usage - Create a clean Git history with conventional commits

What Just Happened?

You built production-ready software in 25 minutes.

Let's count what Claude did autonomously:

Planning: - Analyzed requirements - Chose appropriate libraries - Designed file structure

Implementation: - Created 5+ files - Wrote ~300 lines of code - Implemented CLI arg parsing, file I/O, markdown parsing, syntax highlighting, watch mode

Quality: - Error handling for all edge cases - JSDoc comments - Unit tests - Professional README

Process: - Git initialization - Conventional commits - Version control best practices

Traditional development: This would take 2-4 hours for an experienced developer.

With Claude Code: 25 minutes, and you didn't write a single line of code manually.

The key insight: You focused on **what** you wanted (requirements, features, quality standards), and Claude handled **how** to build it. This is the future of software development — augmented intelligence where humans provide direction and AI provides execution.

Challenge

Go production!

```
Prepare this for npm publish. Add a proper package.json with bin field, add a LICENSE file (MIT), and create a .npmrc for publishing.
```

Then (if you want to actually publish):

```
npm login
npm publish
```

You just built and shipped a CLI tool to npm in under 30 minutes!

Troubleshooting

Common Issues and Solutions

"Claude Code not found"

```
# Reinstall with native installer
curl -fsSL https://claude.ai/install.sh | bash
# Then restart your terminal
```

"Authentication failed"

```
claude logout
claude login
```

Check that you have an active Anthropic subscription with API access.

"Context too large"

```
/compact # Compress conversation history
# Or start fresh:
/clear # Clear conversation but keep file context
```

"Claude not following conventions" - Verify `CLAUDE.md` is in the project root (not a subdirectory) - Check the file is named exactly `CLAUDE.md` (case-sensitive on Linux/Mac) - Try `/clear` and re-run your prompt so Claude re-reads CLAUDE.md

"npm start fails"

```
# Check for port conflicts
lsof -i :3000
# Kill the conflicting process
kill -9 <PID>
```

"Git commit fails"

```
# Ensure Git is configured
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

"Slow responses"

```
/model sonnet    # Use Claude 3.5 Sonnet (faster)
/model haiku      # Use Claude 3 Haiku (fastest)
```

"Claude wants to edit wrong files"

Use @ mentions to focus:

```
@src/routes/todos.js fix the validation logic
```

"Watch mode in Exercise 5 doesn't work"

Ensure chokidar was installed:

```
npm list chokidar
# If missing:
npm install chokidar
```

Key Takeaways

- 1. The Agentic Loop** - Claude doesn't just complete code — it plans, executes, and validates - You provide intent, Claude provides implementation - This is fundamentally different from autocomplete
- 2. CLAUDE.md is Your Team Brain** - One file that teaches Claude your conventions - Automatic adherence to standards - Living documentation that AI and humans both read
- 3. Context Control** - @ mentions = surgical precision - Focus Claude's attention, save tokens - Bring in external docs on-demand

- 4. Natural Language Workflows** - Git, testing, documentation — all in natural language - Stay in flow, don't context-switch to commands - Better commit messages and PR descriptions
 - 5. Speed Matters** - Production-ready CLI in 25 minutes - Focus on WHAT, let Claude handle HOW - Iterate quickly, ship faster
-

Next Steps

Practice Projects: 1. Build a REST API with a real database (PostgreSQL + Prisma) 2. Create a React dashboard with real-time updates 3. Build a CI/CD pipeline with GitHub Actions

Advanced Techniques: - Multi-file refactoring with @ mentions - Custom CLAUDE.md templates for different project types - Using `/compact` strategically in long sessions - Combining Claude Code with other tools (Cursor, GitHub Copilot)

Share Your Work: - Tweet your builds with #ClaudeCode - Write about your experience - Contribute to open source using Claude Code

© 2026 AIA Copilot | Claude Code Fundamentals — Day 2 Lab

For support: scott@aiacopilot.com

Generated from Day2_Lab_Claude_Code_Fundamentals.md on Day2_Lab_Claude_Code_Fundamentals.pdf

© 2026 AIA Copilot Training — Claude Code Fundamentals